

# Lab 4: Macro and Subroutine

- **Link**

- [Document](#)
- [Video](#)

- **Basic (50%)**

- **Description**

Your task is to develop a macro that performs the following sequence of operations:

1. Accept four 8-bit unsigned integers as input:
  - x1 is stored at memory address **[0x000]**.
  - x2 is stored at memory address **[0x001]**.
  - y1 is stored at memory address **[0x002]**.
  - y2 is stored at memory address **[0x003]**.
2. Multiplies x1 with x2, and y1 with y2.
3. Since each multiplication involves two 8-bit unsigned values, each product must be treated as a 16-bit unsigned value. The two 16-bit products must then be added together using 16-bit addition.
4. Stores the final 16-bit sum at memory addresses 0x010 and 0x011:
  - The high byte at address **[0x010]**.
  - The low byte at address **[0x011]**.

You may assume that the final addition does not overflow the 16-bit unsigned range.

- **Example**

| Address        | Test Case 1 | Test Case 2 |
|----------------|-------------|-------------|
| <b>[0x000]</b> | 0xFF        | 0xDC        |
| <b>[0x001]</b> | 0x02        | 0xA0        |
| <b>[0x002]</b> | 0x80        | 0xB0        |
| <b>[0x003]</b> | 0x03        | 0x30        |
| <b>[0x010]</b> | 0x03        | 0xAA        |
| <b>[0x011]</b> | 0x7E        | 0x80        |

- **Standard of grading**

1. The macro must be defined and invoked in the form **Mul\_Add x1, x2, y1, y2**.
2. The two multiplication results and their 16-bit sum must be computed correctly, and the final high-order and low-order bytes must be stored at

memory addresses **0x010** and **0x011**, respectively.

- **Advance (30%)**

- **Description**

Your task is to reverse the order of the four nibbles (hexadecimal digits of a 16-bit unsigned integer using a 16-bit unsigned division subroutine:

1. Accept a 16-bit unsigned integer as input from memory:
  - The high byte is stored at memory address **[0x000]**.
  - The low byte is stored at memory address **[0x001]**.
2. You must implement and call a subroutine named **"division"** that performs 16-bit unsigned division.
  - Hint: The nibble-reversal procedure must use this subroutine. Since dividing a value by 16 produces the least significant hexadecimal digit as the remainder and the remaining higher-order digits as the quotient, repeated division by 16 can be used to extract the four nibbles of the original value.
3. After all four nibbles have been extracted, reconstruct the 16-bit value in reversed nibble order. All four nibbles must be processed, including zero-valued nibbles. For example, 0x1200 must be reversed to 0x0021.
4. Stores the final reversed 16-bit result in memory:
  - The high byte at address **[0x010]**.
  - The low byte at address **[0x011]**.

- **Example**

| Address        | Test Case 1 | Test Case 2 |
|----------------|-------------|-------------|
| <b>[0x000]</b> | 0x12        | 0x3C        |
| <b>[0x001]</b> | 0x34        | 0x7B        |
| <b>[0x010]</b> | 0x43        | 0xB7        |
| <b>[0x011]</b> | 0x21        | 0xC3        |

- **Standard of grading**

1. Your program must contain a subroutine named division, and the nibble-reversal procedure must explicitly use this subroutine.
2. The program must correctly reverse all four nibbles of the 16-bit input value and store the final result at memory addresses 0x010 and 0x011 in high-byte-first order.

- **Hard (20%)**

- **Description**

Implement a Fraction Calculator that performs arithmetic operations on fractions using unsigned integer arithmetic.

Each fraction is represented as:

$$\frac{A}{B}$$

where both the numerator (A) and denominator (B) are **8-bit unsigned integers**. The denominator must be nonzero.

Your program must implement a subroutine named **"fractionCalc"** that performs **one arithmetic operation between two fractions**.

The supported operations are:

- Addition:  $\frac{A}{B} + \frac{C}{D}$
- Subtraction:  $\frac{A}{B} - \frac{C}{D}$
- Multiplication:  $\frac{A}{B} \times \frac{C}{D}$
- Division:  $\frac{A}{B} \div \frac{C}{D}$

The result of each fraction operation must be reduced to its **simplest form** before it is used in a subsequent calculation. To perform this reduction, you must implement another **GCD (Greatest Common Divisor) subroutine** that computes the greatest common divisor of the numerator and denominator.. A fraction is considered to be in its simplest form when

$$GCD(numerator, denominator) = 1.$$

Each test case will contain **three fractions and two operations**, in the following format:

$$\frac{A}{B} \text{ op1 } \frac{C}{D} \text{ op2 } \frac{E}{F}$$

The complete expression must be evaluated according to standard arithmetic precedence. Multiplication and division have higher precedence than addition and subtraction. If two operators have the same precedence, they must be evaluated from left to right. For example,  $A/B + C/D \times E/F$  must be evaluated as  $A/B + (C/D \times E/F)$ , whereas  $A/B - C/D + E/F$  must be evaluated from left to right.

The input fractions and the selected operation are stored in the following memory locations:

- The numerator (A) is stored at address **[0x000]**.
- The denominator (B) is stored at address **[0x001]**.
- The numerator (C) is stored at address **[0x002]**.
- The denominator (D) is stored at address **[0x003]**.
- The numerator (E) is stored at address **[0x004]**.
- The denominator (F) is stored at address **[0x005]**.

- The op1 is stored at address [0x006].
- The op2 is stored at address [0x007]

The **operation code** is defined as follows:

| Operation      | Code |
|----------------|------|
| Addition       | 0x00 |
| Subtraction    | 0x01 |
| Multiplication | 0x02 |
| Division       | 0x03 |

After the complete expression has been evaluated and reduced to its simplest form, the final numerator and denominator must be stored as 16-bit unsigned values. Store the high-order byte of the numerator at address [0x010] and its low-order byte at [0x011]. Store the high-order byte of the denominator at address [0x012] and its low-order byte at [0x013].

For subtraction, the provided test cases are guaranteed to produce a non-negative and nonzero result. For division, the fraction used as the divisor is guaranteed to be nonzero; therefore, its numerator will not be zero. All intermediate and final values required by the provided test cases will fit within the 16-bit unsigned range, and no zero denominator will occur during the calculation.

■ **Example**

| Address | Test Case 1 | Test Case 2 |
|---------|-------------|-------------|
| [0x000] | 0xC7        | 0x71        |
| [0x001] | 0xE9        | 0x1B        |
| [0x002] | 0x11        | 0xB4        |
| [0x003] | 0x05        | 0xC4        |
| [0x004] | 0x03        | 0xF3        |
| [0x005] | 0x28        | 0x31        |
| [0x006] | 0x00        | 0x01        |
| [0x007] | 0x02        | 0x03        |

|         |      |      |
|---------|------|------|
| [0x010] | 0xC9 | 0x00 |
| [0x011] | 0xE3 | 0x04 |
| [0x012] | 0xB6 | 0x00 |
| [0x013] | 0x08 | 0x01 |

- **Standard of grading**
  1. There must be two **subroutines named "fractionCalc" and "GCD."** The fractionCalc subroutine must be used to perform the required fraction operations, while the GCD subroutine must be used to reduce the resulting fractions to their simplest form.
  2. The expression must be evaluated according to the arithmetic precedence specified above. The final reduced numerator and denominator must be computed correctly and stored at memory addresses 0x010 through 0x013 in the specified high-byte and low-byte order.
- Note: If you have any questions, please send an email to the TA mailbox (mprocessor2026@gmail.com). Emails sent to individual TAs will not be answered.